Infrastructure / SRE Intern Assignment - Zeotap

Name: Piyush Bhardwaj
Project: Mission-Critical Incident Management System (IMS)
GitHub Repository: https://github.com/piyushxbhardwaj/Incident-Management-System

---

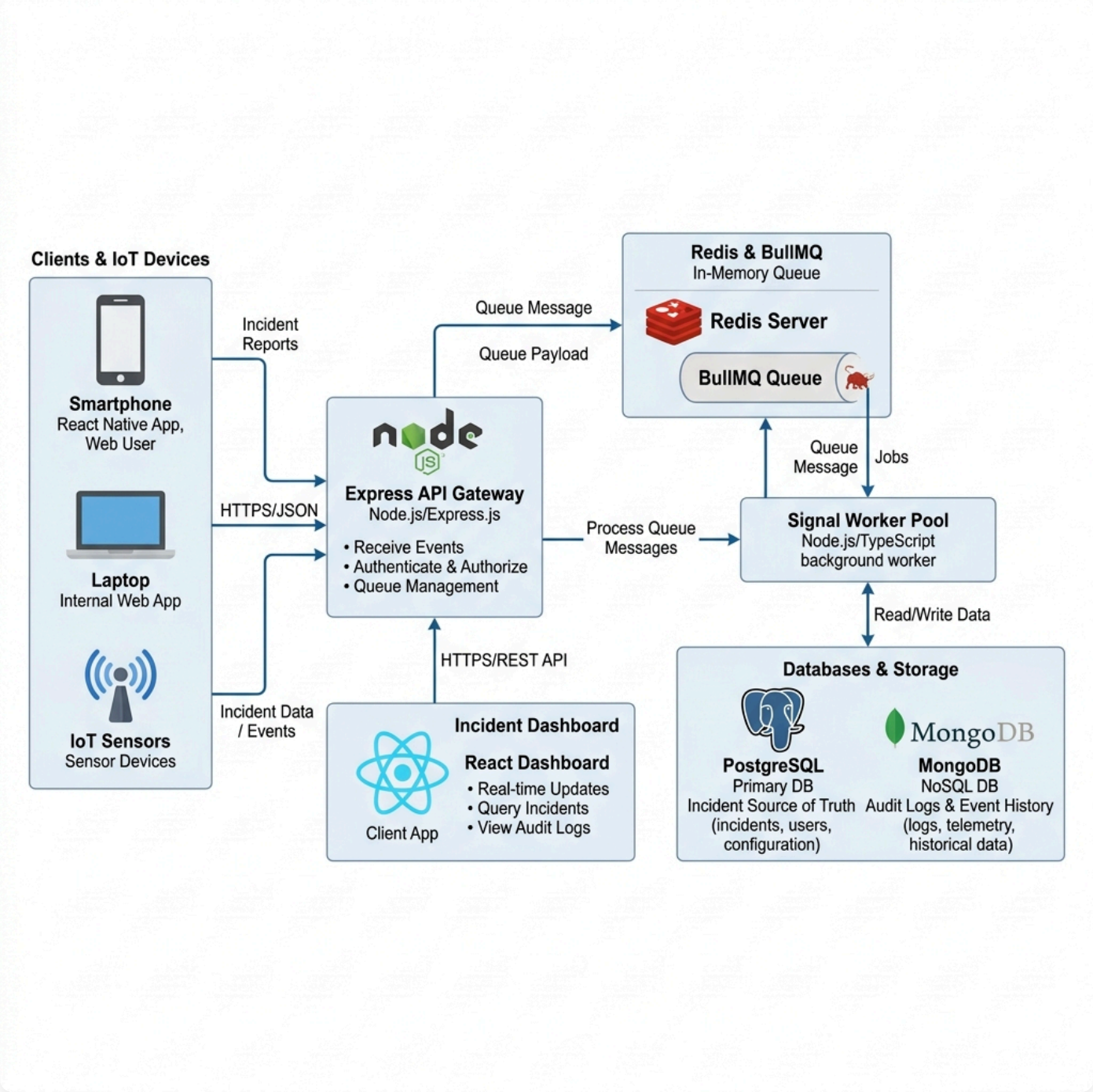
1. Project Overview

This Incident Management System (IMS) is architected for **resilience and high throughput**, designed to monitor complex distributed stacks. It handles high-volume error signals (up to 10,000/sec) through an asynchronous, event-driven pipeline.

Core Stack:

- **Backend:** Node.js (Express)
- **Async Queue:** BullMQ with Redis
- **Data Lake (NoSQL):** MongoDB (Raw audit logs)
- **Source of Truth (RDBMS):** PostgreSQL (Structured incidents & RCA)
- **Frontend:** React Dashboard

🏗️ Architecture Diagram



2. Technical Architecture & Non-Functional Features

🚀 High-Throughput & Backpressure Handling

The system decouples ingestion from persistence using a **Producer-Consumer model**.

- **Ingestion:** API accepts signals and instantly pushes them to a Redis-backed queue.
- **Consumer:** A dedicated worker processes signals at a sustainable rate, protecting MongoDB and PostgreSQL from being overwhelmed during traffic bursts.

- **Rate Limiting:** `express-rate-limit` prevents cascading failures by capping requests at 10,000 signals/sec per IP.

🛡️ Security Layer (Bonus)

- **API Key Protection:** The ingestion endpoint (`/api/signals`) is secured with `X-API-KEY` header validation.
- **Dashboard Authorization:** All management APIs are protected via **JWT (Mock)** validation.
- **Environment Management:** Centralized configuration via `.env` files for secure credential handling.

📉 Intelligent Debouncing

To prevent "alert fatigue," the system implements a **10-second sliding window** count in Redis.

- If **100+ signals** arrive for the same component within 10s, only one incident is created in PostgreSQL.
- **Linkage:** All individual signals are linked to that specific incident in MongoDB for deep-dive root cause analysis.

🏗️ Design Patterns

- **State Pattern:** Manages the incident lifecycle (OPEN ➔ INVESTIGATING ➔ RESOLVED ➔ CLOSED) with strict transitions.
- **Strategy Pattern:** Swaps alerting logic (P0, P1, P2) based on the component type (e.g., RDBMS failure vs Cache warning).

---

3. Observability & Testing

🩺 Deep Health Checks

The `/health` endpoint performs recursive connectivity checks for MongoDB, PostgreSQL, and Redis, returning a `503` if any core infrastructure is unreachable.

📊 Structured Logging

The system uses **JSON-formatted structured logs** for production-grade auditability and easy integration with ELK or Datadog.

🧪 Automated Validation

- **Unit Tests:** Validates the State Machine and RCA enforcement.
- **Integration Tests:** A full-pipeline test script that simulates a failure burst and verifies the end-to-end flow from API to PostgreSQL.

---

4. Setup & Running Instructions

1. **Infra:** `docker-compose up -d`
2. **Backend:** `cd backend && npm start`
3. **Worker:** `cd backend && node src/workers/signalWorker.js`
4. **Frontend:** `cd frontend && npm start`
5. **Test:** `cd backend && npm run test:integration`

---

5. Summary of Engineering Highlights

- **Mandatory RCA:** Prevents closing incidents without root cause documentation.
- **MTTR Calculation:** Automatically calculates the Mean Time to Repair.
- **Transactional Integrity:** State transitions are validated against a strict state machine before DB updates.

Piyush Bhardwaj
Infrastructure / SRE Intern Candidate